

AI STUDYMATE

AI-Powered Student Study Assistant

AI Project Design and Development

Core AI Project Pipeline

INPUT → AI LOGIC → INTERFACE → STORAGE → ACTION

Item	Details
Project Type	AI-powered educational assistant
Primary Interface	Gradio web interface
AI Approach	Semantic retrieval + Retrieval-Augmented Generation (RAG)
Embedding Model	Sentence Transformers — all-MiniLM-L6-v2
Vector Search	FAISS with normalized embeddings / inner-product similarity
Generative Model	Google Gemini 3.6 Flash
Storage	JSON-based study history
Development Environment	Google Colab / Python

1. Introduction

AI StudyMate is an AI-powered student study assistant designed to help students interact with their own lecture notes and study material. The system accepts PDF or TXT study material, converts the content into searchable semantic representations, retrieves the most relevant passages for a student's question, and uses a generative AI model to produce a grounded response. In addition to question answering, the system can generate summaries and quizzes from the uploaded material.

The project demonstrates a complete AI application pipeline rather than only a machine-learning model. It combines data input, AI processing, a user interface, persistent storage, and useful actions in one Python-based Google Colab notebook.

2. Problem Statement

Students often have large amounts of lecture material but need quick answers to specific questions, concise summaries, and practice questions. Traditional keyword search may fail when the student's wording differs from the wording in the notes. AI StudyMate addresses this problem through semantic retrieval, allowing the system to find conceptually relevant passages before generating a response.

3. Proposed Solution

AI StudyMate provides a document-grounded study assistant. A student uploads study material and asks a question. The application extracts the document text, cleans it, divides it into manageable chunks, converts the chunks into embeddings, and indexes them in FAISS. When the student asks a question, the same embedding model represents the question, FAISS retrieves the most relevant chunks, and Gemini generates a response using those retrieved passages as context.

4. Objectives

- Build a complete AI application demonstrating Input → AI Logic → Interface → Storage → Action.
- Allow students to upload PDF or TXT study material.
- Use semantic similarity rather than simple keyword matching for document retrieval.
- Generate grounded answers using Retrieval-Augmented Generation (RAG).
- Provide automated summaries and quizzes from uploaded study material.
- Store study sessions and generated outputs for later reference.
- Provide a simple browser-based interface that can be demonstrated directly from Google Colab.

5. Target Users

The primary target users are university and college students who need a convenient way to study from their own notes, lecture slides, readings, and revision material.

- Students preparing for quizzes, midterms, finals, and assignments.
- Students who want explanations based on a particular course document.
- Students who want rapid summaries and self-assessment quizzes.

6. Use Case

1. The student opens the AI StudyMate web interface.
2. The student uploads a PDF or TXT file containing study material.
3. The system extracts and processes the document.
4. The student asks a question about the uploaded material.
5. The system retrieves relevant document chunks using FAISS.
6. Gemini generates an answer using the retrieved context.
7. The student can additionally generate a summary or quiz.
8. The session and generated content can be stored and downloaded.

7. AI Project Pipeline

Required Pipeline

INPUT → AI LOGIC → INTERFACE → STORAGE → ACTION

Pipeline Component	Implementation in AI StudyMate
Input	PDF/TXT study material and student questions.
AI Logic	Text extraction, cleaning, chunking, Sentence Transformer embeddings, FAISS semantic retrieval, and Gemini RAG.
Interface	Gradio browser-based interface.
Storage	study_history.json for persistent study-session history; in-memory state for the active document/index.
Action	Answer questions, generate summaries, generate quizzes, view history, and download generated outputs.

8. Input Component

The input layer accepts study documents and natural-language questions. PDF files are processed with a PDF text extraction library, while TXT files are read directly. The interface also accepts user commands for question answering, summarization, and quiz generation.

Input	Purpose
PDF	Lecture notes, slides, readings, or other study documents.
TXT	Plain-text study material.
Student question	Natural-language query about the uploaded material.
Study actions	Requests for a summary or quiz.

9. AI Logic

9.1 Text Extraction and Cleaning

The system extracts text from each uploaded document and performs basic cleaning to remove unnecessary whitespace and normalize the text. The cleaned text becomes the basis for semantic retrieval.

9.2 Chunking

Long documents are divided into smaller overlapping text chunks. Chunking makes retrieval more focused because the vector database can return specific portions of the document rather than the entire document at once.

9.3 Embeddings

AI StudyMate uses the pretrained Sentence Transformer model all-MiniLM-L6-v2 to convert document chunks and user questions into numerical vectors. These vectors capture semantic relationships between pieces of text.

9.4 FAISS Semantic Search

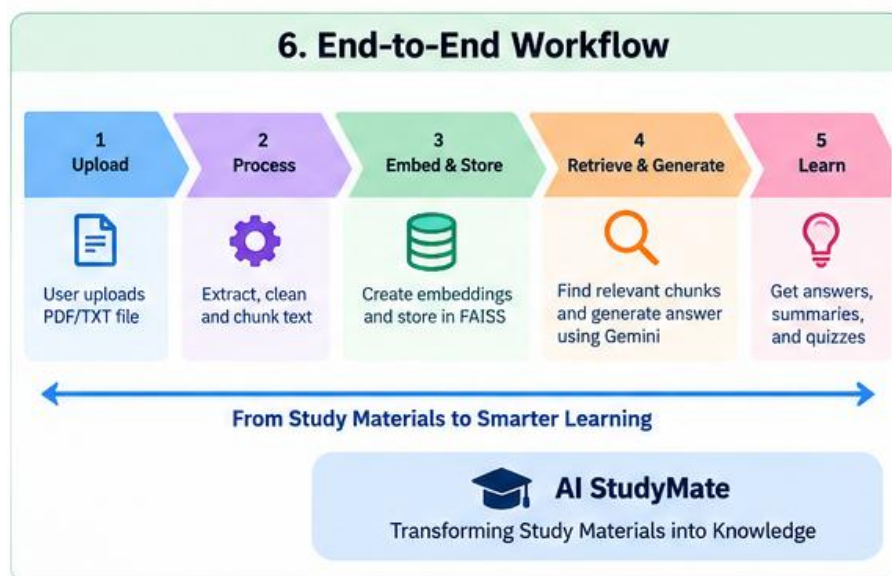
The document embeddings are stored in a FAISS index. The implementation normalizes embeddings and uses an inner-product index, which corresponds to cosine similarity for normalized vectors. The question embedding is compared with indexed document embeddings and the highest-scoring chunks are retrieved.

9.5 Retrieval-Augmented Generation

The retrieved chunks are supplied to Google Gemini as context. Gemini is instructed to answer using the provided study material, making the response more closely tied to the uploaded document.

9.6 Summary and Quiz Generation

The same document-grounded workflow supports concise study summaries and practice quizzes.



10. Interface

The interface is implemented with Gradio and runs directly from Google Colab. It provides document upload, document processing, question answering, retrieved-source display, summary generation, quiz generation, history viewing, and download functionality.

Interface Feature	User Function
Document upload	Select PDF/TXT study material.
Process Document	Extract, clean, chunk, embed, and index the document.
Ask AI	Submit a question and receive a document-grounded answer.
Retrieved Sources	Inspect the document passages used for retrieval.
Generate Summary	Create a concise study summary.
Generate Quiz	Create practice questions from the material.
History	View stored study sessions and generated content.
Download	Save answers, summaries, or quizzes for offline use.

11. Storage

AI StudyMate uses two levels of storage. The active document, chunks, embeddings, and FAISS index are maintained in memory during the running session. Persistent study history is stored in `study_history.json`. A stored session can include document information, timestamps, chat history, the latest summary, and the latest quiz.

Storage Design

Active AI state → In-memory Python objects / FAISS index

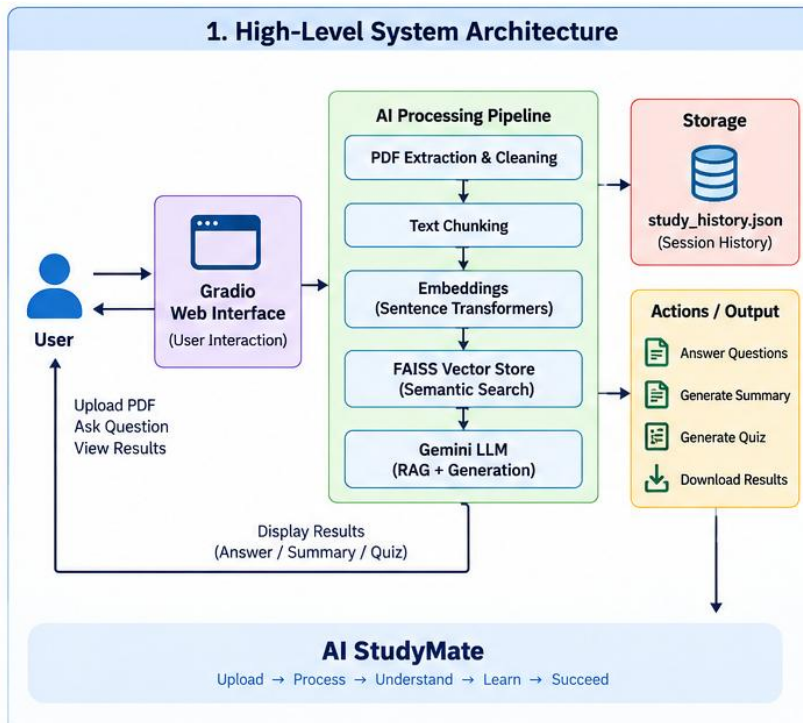
Persistent history → `study_history.json`

12. Action Component

The action layer converts AI outputs into useful student activities. The application can answer questions, produce summaries, generate quizzes, store the study session, display previous history, and prepare downloadable outputs.

13. System Architecture

The complete architecture is:



14. Technology Stack

Technology	Role
Python	Main programming language.
Google Colab	Development and execution environment.
Gradio	Web-based user interface.
Google Gemini API	Generative AI for RAG answers, summaries, and quizzes.
Sentence Transformers	Semantic text embeddings.
all-MiniLM-L6-v2	Pretrained sentence embedding model.
FAISS	Vector indexing and semantic similarity retrieval.
PyPDF	PDF text extraction.
NumPy / pandas	Data and numerical processing where required.
JSON	Persistent study history storage.

15. Implementation

The complete implementation is contained in a single Google Colab notebook. This design keeps the project easy to execute, demonstrate, and submit. No model was trained from scratch; pretrained components are integrated into a complete AI application.

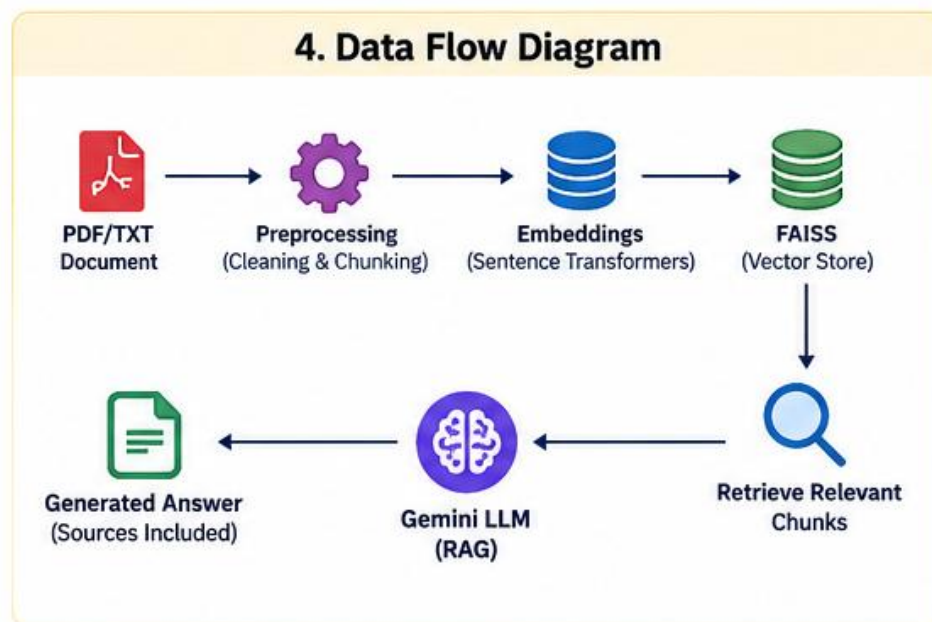
15.1 Core Processing Flow

9. Install and import required Python libraries.
10. Configure the Gemini API client using a runtime API key.
11. Load all-MiniLM-L6-v2.
12. Extract text from the uploaded PDF/TXT document.
13. Clean and split the document into chunks.
14. Generate normalized embeddings for the chunks.
15. Create the FAISS index.
16. Embed the student's question and retrieve relevant chunks.
17. Send retrieved context and the question to Gemini.
18. Display the response through Gradio.
19. Store the session and provide summary, quiz, and download actions.

16. API Key and Security Considerations

The Gemini API key is entered at runtime using password-style input rather than being hard-coded. The key should never be committed to GitHub, included in screenshots, or shared publicly.

- Use getpass or another secure runtime input method.
- Do not place a real API key directly in source code.
- Do not upload a file containing the API key to GitHub.
- If a key is accidentally exposed, revoke it and generate a replacement.



17. Testing and Validation

The completed notebook and Gradio application were executed successfully. Testing covered both individual components and end-to-end user behavior.

Test Case	Expected Result	Status
Gemini API test	Gemini returns a generated response.	Passed
Model listing	Available Gemini models can be queried.	Passed
PDF upload	PDF is accepted by the interface.	Passed
Document processing	Text is extracted, chunked, embedded, and indexed.	Passed
Direct question	Relevant answer is generated from study material.	Passed
Specific question	Relevant semantic context is retrieved.	Passed
Out-of-context question	Absent material is not treated as document evidence.	Passed
Summary generation	Summary is produced from study material.	Passed
Quiz generation	Quiz is generated from study material.	Passed
History	Study history is stored and displayed.	Passed
Download actions	Generated outputs can be prepared for download.	Passed
End-to-end Gradio test	Complete workflow runs through the interface.	Passed

Testing Outcome

The complete AI StudyMate notebook and Gradio interface were run successfully. The core pipeline from document input through semantic retrieval and Gemini generation was operational during testing.

18. Results

The project successfully demonstrated a complete AI application rather than an isolated model. A user can provide study material, process it, ask questions in natural language, retrieve semantically relevant passages, and receive a generated answer. The same interface supports summaries and quizzes while maintaining JSON-based history.

Requirement	Implementation Result
Input	PDF/TXT documents and natural-language questions.
AI Logic	Pretrained embeddings + FAISS retrieval + Gemini RAG.
Interface	Interactive Gradio application.
Storage	In-memory active state + JSON history.
Action	Answers, summaries, quizzes, history, downloads.

19. Limitations

- Answer quality depends on the quality and completeness of the uploaded document.
- Complex, scanned, image-heavy, or table-heavy PDFs may not extract perfectly without OCR.
- The system requires internet access for Gemini and remote model resources.
- Gemini usage is subject to account, model, quota, and service limits.
- The storage design is intended for a lightweight academic prototype rather than a multi-user production system.
- The FAISS index is session-based rather than a permanent multi-user vector database.

20. Future Improvements

- Add OCR for scanned lecture notes and image-based PDFs.
- Support multi-document course collections.
- Replace JSON with SQLite or a cloud database.
- Add authentication and separate user profiles.
- Add page-number citations for generated answers.
- Add adaptive quizzes based on performance.
- Add voice input and text-to-speech.
- Deploy as a hosted web application.
- Add quantitative retrieval and answer-faithfulness evaluation.

21. Conclusion

AI StudyMate demonstrates how modern AI components can be combined into a practical student-focused application. The project satisfies the complete AI application pipeline of Input, AI Logic, Interface, Storage, and Action. By combining semantic retrieval with Retrieval-Augmented Generation, the system can work with user-provided study material rather than functioning only as a general chatbot. The single-notebook implementation makes the prototype easy to demonstrate, test, and submit.

22. GitHub Submission Structure

Recommended repository structure:

```
AI-StudyMate/  
├── AI_StudyMate_Complete_Colab.ipynb  
├── AI_StudyMate_Formal_Project_Document.docx  
├── README.md  
├── .gitignore  
└── study_history.json (optional; avoid uploading personal study data)
```

GitHub Security

Do not upload the real Gemini API key. The notebook should request the key at runtime. If study_history.json contains personal or private material, keep it out of the repository.

23. Demonstration Procedure

20. Open the Google Colab notebook.
21. Install dependencies and run setup cells.
22. Enter a valid Gemini API key when prompted.
23. Run the notebook through the Gradio launch cell.
24. Open the Gradio interface link.
25. Upload a PDF or TXT study document.
26. Click Process Document.
27. Ask a question from the uploaded material.
28. Inspect retrieved sources and the generated answer.
29. Generate a summary and quiz.
30. Open study history and test download actions.

Appendix A — Quick Project Summary

Category	Summary
Project	AI StudyMate — AI-Powered Student Study Assistant
Problem	Students need fast, document-grounded help with study material.
Input	PDF/TXT + natural-language question
AI	Sentence Transformer + FAISS + Gemini RAG
Interface	Gradio
Storage	In-memory state + study_history.json
Actions	Q&A, summary, quiz, history, downloads
Environment	Google Colab
Implementation	Single Python notebook
Status	Implemented and tested successfully

Appendix B — RAG Pipeline

